

INTRODUÇÃO AO VUE.JS

Disciplina: CIC0198 - TECNICAS DE PROGRAMAÇÃO 2

Professor: Daniel Porto



<https://vuejs.org/>

1 Conceitos

1.1 O que é frontend?

Frontend = HTML + CSS + JavaScript

- ▶ **HTML** é o esqueleto/estrutura
- ▶ **CSS** é a maquiagem (passa um batom na estrutura)
- ▶ **JavaScript** é a programação

1.2 O que é o Vue.js?

- ▶ Uma biblioteca que organiza o JavaScript para fazer sites interativos.
- ▶ Permite criar componentes reutilizáveis.
- ▶ Exibe/atualiza conteúdo automaticamente quando os dados mudam.
- ▶ Criador: Evan You - Google

1.3 O que vamos precisar?

Node.js

O Node.js é um ambiente de execução do código JavaScript no computador (não só no navegador). Usamos ele para preparar e rodar projetos web modernos.

Link: <https://nodejs.org>

Node.js (versão LTS) e npm instalados. Verificar com:

```
node -v  
npm -v
```

Vite

Vite é uma ferramenta que cria e roda projetos web modernos rapidamente. Ele já configura tudo pra gente (tipo o "starter pack").

Editor de texto

Use o da sua preferência!

- ▶ VS Code 🐼
- ▶ Webstorm 🍌

- ▶ Fleet 🍌
- ▶ Zed 🍌
- ▶ Bloco de notas 🍌 }...
- ▶ vi 💀

2 Hello World com Vite

1. Abra o terminal.
2. Crie o projeto (duas formas — use a que funcionar no seu ambiente):

```
# opção A (comando + moderno)
npm create vite@latest hello-vue -- --template vue

# ou opção B (se a A não funcionar)
npm init vite@latest hello-vue -- --template vue
```

3. Entre na pasta e instale dependências:

```
cd hello-vue
npm install
```

4. Rode o servidor de desenvolvimento:

```
npm run dev
```

5. Abra no navegador o endereço mostrado. Geralmente `http://localhost:5173`.

OBS: Vue CLI

Vue CLI é uma ferramenta de linha de comando que cria um projeto Vue completo e configurado, usando Webpack.

- ▶ CLI = Command Line Interface
- ▶ Diferente do Vite, o CLI configura Webpack, que é mais poderoso, mas mais complexo
- ▶ Útil para projetos grandes ou quando for necessário configurar coisas como Babel, PWA, TypeScript, etc.

Resumo:

Recurso	Vite	Vue CLI
Velocidade	Extremamente rápido	Mais lento (Webpack)
Complexidade	Simples, minimalista	Mais configurável
Início	<code>npm create vite@latest</code>	<code>vue create meu-projeto</code>
Ideal para	Projetos pequenos/médios	Projetos maiores/complexos

3 Estrutura mínima que vamos usar

Na pasta src/ você encontrará:

- ▶ main.js – ponto de entrada que cria a app.
- ▶ App.vue – componente raiz.
- ▶ components/ – aqui colocaremos todos os nossos componentes.

4 Meu primeiro Componente

Vamos criar o arquivo “MeuComponente.vue” dentro da pasta src/components. Os componentes possuem 3 blocos:

- ▶ Marcação
- ▶ Estilo
- ▶ Comportamento

A estrutura padrão de um componente Vue é:

```
<template>
</template>

<style>
</style>

<script>
</script>
```

Edite o seu arquivo MeuComponente.vue:

```
<template>
  <div>
    <h1>Olá mundo Vue.js</h1>
    <br>
    <p>Esse é o meu primeiro componente!</p>
  </div>
</template>
<style>
</style>
<script setup>
</script>
```

Altere o seu arquivo App.vue para esse conteúdo:

```
<script setup>
import meuComponente from './components/MeuComponente.vue'
</script>

<template>
  <meuComponente/>
</template>
```

Salve tudo. Sua aplicação irá recarregar tudo automaticamente! 😊

5 v-model e interpolação

Agora vamos aprender a usar o ref, v-model e interpolação {{ }}. Abra src/components/MeuComponente.vue e substitua o conteúdo por:

```

<template>
  <div>
    <h1>Olá mundo Vue.js</h1>
    <br>
    <p>Esse é o meu primeiro componente!</p>
    <br>
    <input v-model="mensagem"/>
    <p>Mensagem: <strong>{{ mensagem }}</strong></p>
  </div>
</template>
<style>
</style>
<script setup>
import { ref } from 'vue'
const mensagem = ref('E aí? Qual é a a boa?')
</script>

```

- ▶ ref cria uma variável reativa (mensagem.value no JS, mas no template só mensagem).
- ▶ v-model liga o valor do input à variável.

Extra

- ▶ Trocar input por textarea.
- ▶ Mostrar o número de caracteres com {{ mensagem.length }} (use computed em uma segunda versão).

6 Events e Binds

Vamos brincar agora de @click, métodos e manipulação de ref. Para isso, vamos criar um contador. Crie o arquivo src/components/Counter.vue:

```

<template>
  <div class="counter">
    <h2>Contador</h2>
    <p class="value">{{ count }}</p>
    <div class="controls">
      <button @click="decrement">-</button>
      <button @click="increment">+</button>
      <button @click="reset">Reset</button>
    </div>
  </div>
</template>

<script setup>
import { ref } from 'vue'
const count = ref(0)

function increment() { count.value++ }
function decrement() { count.value-- }
function reset() { count.value = 0 }
</script>

<style scoped>
.counter { text-align: center; }
.value { font-size: 2rem; margin: 8px 0; }
.controls button { margin: 0 6px; padding: 6px 12px; }
</style>

```

Use no App.vue – importe e use o componente:

```

<script setup>
import meuComponente from './components/MeuComponente.vue'
import conter from './components/Counter.vue'

```

```

</script>

<template>
  <conter/>
  <meuComponente/>
</template>

```

Extra

- Use a diretiva condicional v-if para habilitar ou não os botões, dependendo do valor do contador.

7 Lista de Tarefas (To-Do) — CRUD básico

Opa. Agora vamos aprender um pouco de v-for, v-bind:key, adicionar/remover itens, e persistência simples com localStorage. Vamos criar o nosso arquivo `TodoList.vue`

```

<template>
  <div class="todo">
    <h2>Lista de Tarefas</h2>

    <form @submit.prevent="addTodo">
      <input v-model="novo" placeholder="Nova tarefa..." />
      <button type="submit">Adicionar</button>
    </form>

    <ul>
      <li v-for="todo in todos" :key="todo.id">
        <label>
          <input type="checkbox" v-model="todo.done" />
          <span :class="{ done: todo.done }">{{ todo.text }}</span>
        </label>
        <button @click="removeTodo(todo.id)">Remover</button>
      </li>
    </ul>

    <p v-if="todos.length === 0">Nenhuma tarefa por enquanto </p>
  </div>
</template>

<script setup>
import { ref, watch, onMounted } from 'vue'

const novo = ref('')
const todos = ref([])

// carregar do localStorage
onMounted(() => {
  const raw = localStorage.getItem('todos')
  if (raw) todos.value = JSON.parse(raw)
})

function addTodo() {
  if (!novo.value.trim()) return
  const item = {
    id: Date.now(),
    text: novo.value.trim(),
    done: false
  }
  todos.value.unshift(item)
  novo.value = ''
}

// remover por id
function removeTodo(id) {
  todos.value = todos.value.filter(t => t.id !== id)
}

```

```
// salvar sempre que mudar (deep)
watch(todos, (nv) => {
  localStorage.setItem('todos', JSON.stringify(nv))
}, { deep: true })
</script>

<style scoped>
.todo { max-width: 600px; margin: 20px auto; }
.done { text-decoration: line-through; color: gray; }
ul { list-style: none; padding: 0; }
li { display: flex; align-items: center; justify-content: space-between; padding: 8px 0; }
</style>
```

Extra

- ▶ Adicionar filtro (Todas, Ativas, Completas).
- ▶ Ordenar por data ou por status.

8 Comunicação Pai ↔ Filho (props e emits)

No Vue, o pai passa valores para o componente filho. O filho recebe esses valores via props. Os props são como os parâmetros de uma função. Exemplo rápido: Vamos criar um componente filho que é responsável por renderizar apenas 1 item da lista. Esse componente recebe o item via props e emite eventos ('toggle' e 'remove') para avisar o pai para mudar o valor do objeto. `TodoItem.vue` (filho)

```
<template>
  <li>
    <label>
      <input type="checkbox" :checked="todo.done" @change="$emit('toggle', todo.id)" />
      <span :class="{ done: todo.done }">{{ todo.text }}</span>
    </label>
    <button @click="$emit('remove', todo.id)">Remover</button>
  </li>
</template>

<script setup>
const props = defineProps({ todo: Object })
</script>
```

No pai (`TodoList.vue`) você agora usa:

```
<TodoItem
  v-for="todo in todos"
  :key="todo.id"
  :todo="todo"
  @toggle="updateTodo"
  @remove="removeTodo"
/>
```

E temos que definir a função `updateTodo`.

```
// atualizar item quando mudar no filho
function updateTodo(updated) {
  const idx = todos.value.findIndex(t => t.id === updated.id)
  if (idx !== -1) {
    todos.value[idx] = { ...updated }
  }
}
```

Você se esqueceu de importar o componente filho dentro do `<script setup>` do pai?

```
import TodoItem from './TodoItem.vue'
```

9 Cartão de Perfil (props e slot básico)

Vamos criar agora um componente ProfileCard.vue

```
<template>
  <div class="card">
    
    <h3>{{ name }}</h3>
    <p v-if="age">Idade: {{ age }}</p>
    <slot name="extra"></slot>
  </div>
</template>

<script setup>
const props = defineProps({
  name: { type: String, required: true },
  age: { type: Number, default: null },
  avatar: { type: String, default: '' }
})
</script>

<style scoped>
.card { border: 1px solid #ddd; padding: 12px; border-radius: 8px; width: 220px; text-align:center; }
.avatar { width: 80px; height: 80px; border-radius: 50%; object-fit: cover; margin-bottom:8px; }
</style>
```

Pra usar ele no App.vue:

```
<profileCard name="João" :age="28" avatar="'https://i.pravatar.cc/150?u=Joao'">
  <template #extra>
    <p>Estudante de Ciência da Computação</p>
  </template>
</profileCard>
```

Você importou seu componente ProfileCard no App.vue?

10 Rotas

O vue-router é o plugin oficial do Vue para criar rotas.

Ele é responsável por criar Single Page Applications com múltiplas páginas/telas usando rotas.

No fundo, ele gerencia a troca de componentes quando a URL muda, sem recarregar a página inteira.

```
npm i vue-router --save
```

Arquivo de rotas

Agora precisamos criar um arquivo de rotas router/router.js:

```
import { createRouter, createWebHistory } from 'vue-router'
import HomeView from '../views/HomeView.vue'
import AboutView from '../views/AboutView.vue'
const routes = [
  { path: '/',
    name: 'home',
    component: HomeView },
  { path: '/about',
    name: 'about',
    component: AboutView}
]
const router = createRouter({
  history: createWebHistory(),
  routes
})
```

```
export default router
```

Registrando o router na aplicação
main.js:

```
import { createApp } from 'vue'
import App from './App.vue'
import router from './router/router'

const app = createApp(App)

app.use(router)
app.mount('#app')
```

Criando os componentes de página
src/views/HomeView.vue:

```
<template>
  <div>
    <h1>Home</h1>
    <p>Bem-vindo à página inicial!</p>
  </div>
</template>
```

src/views/AboutView.vue:

```
<template>
  <div>
    <h1>Sobre</h1>
    <p>Essa é a página Sobre.</p>
  </div>
</template>
```

Exibir os links das páginas
App.vue:

```
<template>
  <nav>
    <router-link to="/">Home</router-link> |
    <router-link to="/about">Sobre</router-link>
  </nav>

  <!-- Aqui o Vue Router renderiza o componente da rota -->
  <router-view></router-view>
</template>
```

Navegando

src/views/HomeView.vue:

```
<template>
  <div>
    <h1>Home</h1>
    <p>Bem-vindo à página inicial!</p>
    <br><br>
    <button @click="irParaSobre()">Vamos para outra página?</button>
  </div>
</template>

<script setup>
import { useRouter } from 'vue-router'
const router = useRouter()
function irParaSobre() {
  router.push('/about')
}
</script>
```

11 Boas práticas

- ▶ Componentes pequenos e com uma única responsabilidade.
- ▶ Nomear componentes PascalCase (MyButton.vue).
- ▶ Usar key em v-for.
- ▶ Evitar manipular o DOM diretamente (prefira reatividade).
- ▶ Documente props (defineProps) e eventos (defineEmits).

12 Erros comuns do iniciante

- ▶ Esquecer .value em ref dentro do script.
- ▶ Não usar :key em v-for (resulta em bugs ao reordenar).
- ▶ Mutar arrays/objetos sem reatividade (prefira todos.value = [...todos.value, novo] ou use reactive).
- ▶ Colocar lógica pesada diretamente no template.

13 E agora?

Agora que você já sabe tudo de Vue.js, dê uma olhada com carinho na documentação oficial do vue: <https://vuejs.org/guide/introduction.html>

Dê uma pesquisada sobre vuex.

Mais para frente... (ou quem for curioso)

- ▶ Karma: <http://karma-runner.github.io/>
- ▶ Mocha: <https://mochajs.org>
- ▶ Chaijs: <http://chaijs.com/>
- ▶ Sinon: <http://sinonjs.org>

14 Referências

- ▶ VILARINHO, Leonardo. Front-end com Vue. js: Da teoria à prática sem complicações. Editora Casa do Código, 2021.
- ▶ INCAU, Caio. Vue. js: Construa aplicações incríveis. Editora Casa do Código, 2017.
- ▶ AU-YEUNG, John. Vue. js 3 By Example. Packt Publishing, 2021.
- ▶ <https://vuejs.org/>